

Docker Swarm — Complete Beginner-to-Advanced Guide

You already know Docker basics, volumes, networks, and Compose. This guide builds on that, step by step, from **theory** → **hands-on commands** → **advanced concepts** → **interview prep**.

Every command block is heavily commented so you understand *why*, not just *what*.

STEP 1: What Problem Does Docker Swarm Solve? (Theory)

So far, everything you've done with Docker runs on **one machine**. `docker run`, `docker compose up` — all single-host.

But in the real world:

- You have **multiple servers** (nodes), not just one.
- If one server dies, your app should keep running (**high availability**).
- If traffic increases, you want to run **more copies** of your app across servers (**scaling**).
- You want traffic automatically spread across all copies (**load balancing**).
- You want to update your app with **zero downtime**.

A single Docker host can't do this — it has no concept of "other machines." You need an **orchestrator**: software that manages containers across a *cluster* of machines.

Docker Swarm is Docker's own built-in orchestrator (an alternative to Kubernetes, but much simpler to learn and set up). It's not a separate install — it's a *mode* built into the Docker Engine you already have.

Kubernetes vs Swarm (just so you know the landscape): Kubernetes is more powerful and is the industry standard for large/complex systems, but has a steep learning curve. Swarm is simpler, uses the same `docker` CLI/Compose syntax you already know, and is great for small-to-medium clusters or learning orchestration concepts. Learning Swarm first makes Kubernetes much easier later, because the core ideas (nodes, services, replicas, load balancing) are the same.

STEP 2: Core Swarm Concepts (Theory)

Before touching the terminal, understand this vocabulary — everything else builds on it.

Term	Meaning
Node	A single machine (physical or VM) running Docker, participating in the swarm.
Manager node	A node that manages the cluster: accepts commands, schedules work, keeps cluster state.
Worker node	A node that only runs containers — it takes orders from managers, doesn't give them.
Service	A <i>definition</i> of how a container should run at cluster scale (image, replicas, ports, network). This replaces plain <code>docker run</code> in Swarm.
Task	A single running container that's part of a service — Swarm's smallest unit of scheduling.
Replica	One copy/instance of a service's task. "5 replicas" = 5 identical containers running.
Stack	A group of services deployed together, defined in a Compose file (like <code>docker compose up</code>) but for the whole cluster).
Overlay network	A virtual network that spans multiple physical hosts, letting containers on different machines talk to each other as if on the same LAN.
Routing Mesh	Swarm's built-in load balancer — lets you hit <i>any</i> node's IP on a published port and get routed to a healthy container, even if it's running on a different node.

Manager vs Worker — how decisions get made

- Managers use a consensus algorithm called **Raft** to agree on the cluster's state (which services should run, how many replicas, etc.) even if some managers go down.
- **Odd numbers of managers** are recommended (1, 3, 5) — Raft needs a majority ("quorum") to agree, and odd numbers avoid tie votes. E.g., with 3 managers, the cluster survives 1 manager failure; with 5, it survives 2.
- Workers **only execute tasks** — they can't make scheduling decisions. This keeps the cluster's "brain" small and consistent.
- A manager node, by default, is *also* a worker (it can run containers too) unless you drain it.

STEP 3: Initializing Your First Swarm (Hands-on)

You don't need multiple machines to learn this — one machine can be a single-node swarm. Later we'll talk about multi-node setups.

```
bash
```

```
# Check your Docker version supports swarm (it's built-in since Docker 1.12+)
```

```
docker --version
```

```
# Turn your current Docker Engine into a Swarm manager.
```

```
# This single command creates a brand-new swarm cluster with THIS machine as its
```

```
docker swarm init
```

```
# If you have multiple network interfaces (common on cloud VMs with private+public
```

```
# tell Docker explicitly which IP other nodes should use to reach this manager:
```

```
docker swarm init --advertise-addr 192.168.1.10
```

What just happened internally?

- Docker generated cryptographic certificates for secure node-to-node communication (TLS).
- This node became a **manager** and the **leader** (in Raft terms).
- Docker printed a `docker swarm join` command with a **worker token** — save this, you'll use it to add more nodes later.

```
bash
```

```
# Confirm you're now in swarm mode and see cluster nodes
```

```
docker node ls
```

```
# Example output:
```

```
# ID                HOSTNAME        STATUS    AVAILABILITY    MANAGER STATUS
# abc123 *          my-machine     Ready     Active           Leader
#                                     ^ this node is the
```

The (*) marks the node you're currently running commands from.

STEP 4: Adding More Nodes (Hands-on — Multi-Machine)

Real swarms span multiple machines. If you're practicing, you can spin up 2-3 VMs (VirtualBox, cloud VMs, or even Docker-in-Docker containers) — each needs Docker installed and network connectivity to the manager.

```
bash
```

```
# On the MANAGER, retrieve the join command/token for WORKERS (if you lost the
docker swarm join-token worker
```

```
# Output looks like:
```

```
# docker swarm join --token SWMTKN-1-xxxxxxxxxxxxx 192.168.1.10:2377
```

```
#           ^ secret token           ^ manager's IP : swarm list
```

```
# On a WORKER machine, run that exact command to join the cluster:
```

```
docker swarm join --token SWMTKN-1-xxxxxxxxxxxxx 192.168.1.10:2377
```

```
# To add another MANAGER (not just a worker), get the manager-specific token in
docker swarm join-token manager
```

```
# Then run the printed command on the machine you want to promote to manager.
```

```
bash
```

```
# Back on the manager, verify all nodes joined:
```

```
docker node ls
```

```
# You should now see multiple rows – some MANAGER STATUS = "Reachable" or "Lead
```

```
# workers will have MANAGER STATUS = empty (they're not part of the management
```

Promoting/demoting nodes — you can change a worker into a manager or vice versa without leaving/rejoining the swarm:

```
bash
```

```
# Promote a worker to manager (gives it decision-making power)
```

```
docker node promote <NODE-NAME>
```

```
# Demote a manager back to worker
```

```
docker node demote <NODE-NAME>
```

STEP 5: Your First Service — Replacing `docker run` (Hands-on)

In Swarm, you don't `docker run` a container directly on a node. Instead, you tell the **swarm** "I want this service to exist," and the swarm decides which node(s) to run it on.

```
bash
```

```
# Create a service running nginx, with 3 identical replicas (copies)
```

```
docker service create \
  --name my-web \
  --replicas 3 \
  --publish published=8080,target=80 \
  nginx:latest
```

```
# --name my-web           : name for this service (used to manage/update/remove)
# --replicas 3            : run 3 containers of this service across the cluster
# --publish published=8080,target=80
#                          : expose container port 80 (nginx's default) as port 8080
#                          : on EVERY node in the swarm (this is the "routing mesh" - see
#                          : more on this in Step 8)
# nginx:latest            : the image to run
```

```
bash
```

```
# See your services
```

```
docker service ls
```

```
# ID            NAME      MODE      REPLICAS  IMAGE          PORTS
# xyz789        my-web    replicated 3/3        nginx:latest   *:8080->80/tcp
#               ^ 3 desired, 3 actually running = healthy
```

```
# See WHICH nodes are running which replicas (the individual tasks/containers)
```

```
docker service ps my-web
```

#	ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE
#	a1b2	my-web.1	nginx:latest	worker-1	Running	Running 2 mins
#	c3d4	my-web.2	nginx:latest	worker-2	Running	Running 2 mins
#	e5f6	my-web.3	nginx:latest	manager-1	Running	Running 2 mins

Now try hitting `http://<ANY-NODE-IP>:8080` from a browser — even a node that ISN'T running a copy of nginx will still route your request correctly. That's the routing mesh doing load balancing for you, automatically.

Self-healing test: kill one of the containers manually and watch Swarm recreate it:

```
bash
```

```
# Pick a node running a replica, and force-remove that container using normal d
docker rm -f <container_id>
```

```
# Now immediately check again:
```

```
docker service ps my-web
```

```
# Swarm noticed the desired count (3) no longer matched running count, and
# scheduled a brand new replica within seconds to fix it. This is the core valu
# of orchestration: the cluster constantly reconciles "desired state" vs "actua
```

STEP 6: Scaling Services (Hands-on)

```
bash
```

```
# Scale up to 6 replicas – Swarm will schedule 3 MORE containers across availab
docker service scale my-web=6
```

```
# Scale multiple services at once
```

```
docker service scale my-web=6 another-service=2
```

```
# Scale back down
```

```
docker service scale my-web=2
```

Swarm automatically decides *where* (which nodes) to place replicas, using a default "spread" strategy — trying to balance load evenly across nodes, unless you add placement constraints (Step 11).

STEP 7: Updating Services with Zero Downtime (Hands-on, Intermediate)

This is a big reason to use Swarm — updating your app without taking it offline.

```
bash
```

```
# Update the image version of a running service
```

```
docker service update --image nginx:1.25 my-web
```

```
# What happens: Swarm doesn't kill all 3 (or 6) containers at once.
```

```
# By default it does a ROLLING UPDATE: stop old container -> start new one -> w
```

```
# repeat for the next replica. Users never see full downtime.
```

You can control the rollout behavior precisely:

```
bash
```

```
docker service update \  
  --image nginx:1.25 \  
  --update-parallelism 1 \  
  --update-delay 10s \  
  --update-order start-first \  
  my-web  
  
# --update-parallelism 1 : update ONE replica at a time (safer, slower)  
# --update-delay 10s    : wait 10 seconds between each replica update  
#                        (gives you time to notice if something's broken)  
# --update-order start-first  
#                        : start the NEW container before stopping the old one  
#                        (avoids any dip in available capacity; default is  
#                        "stop-first" which stops old before starting new)
```

Rolling back if the update breaks something:

```
bash
```

```
# Swarm remembers the previous spec of the service  
docker service rollback my-web
```

Failure handling during updates:

```
bash
```

```
docker service update \  
  --image nginx:1.25 \  
  --update-failure-action rollback \  
  --update-max-failure-ratio 0.2 \  
  my-web  
  
# --update-failure-action rollback  
#                        : if too many replicas fail to start with the new image,  
#                        automatically roll back instead of leaving things broken  
# --update-max-failure-ratio 0.2  
#                        : tolerate up to 20% of replicas failing before triggering  
#                        the failure-action
```

STEP 8: Overlay Networks & the Routing Mesh (Theory + Hands-on, Intermediate)

You already know `docker network create` for single-host bridge networks. Swarm needs networks that span **multiple hosts** — that's an **overlay network**.

```
bash

# Create an overlay network (only works in swarm mode)
docker network create \
  --driver overlay \
  --attachable \
  my-overlay-net

# --driver overlay    : creates a virtual network spanning all swarm nodes
# --attachable       : allows standalone containers (not just services) to also
#                     join this network manually with `docker run --network`
```

```
bash

# Attach a service to this overlay network so its replicas (wherever they run)
# can reach each other and other services by NAME (like Compose's DNS-based discovery)
docker service create \
  --name my-api \
  --network my-overlay-net \
  --replicas 3 \
  my-api-image:latest
```

Any container on `my-overlay-net`, regardless of which physical node it's on, can reach `my-api` by that name — Docker's internal DNS resolves it and load-balances across replicas (this is called the **VIP — Virtual IP** mode by default).

The Routing Mesh (Ingress Network)

When you `--publish` a port on a service (like Step 5), Swarm automatically:

1. Creates a special built-in overlay network called `ingress`.
2. Opens that port on **every single node**, even ones not running a replica.
3. Internally routes any incoming request on that port to a healthy container, *anywhere in the cluster*, using IPVS (Linux kernel load balancing).

This means your users/load-balancer can point to **any node's IP** and always reach your service. You don't need to know which node is "actually" running it.

```
bash

# Inspect the auto-created ingress network
docker network ls
docker network inspect ingress
```


STEP 9: Deploying with Stacks (Compose Files in Swarm) (Hands-on, Intermediate)

You already know Compose files. In Swarm, you deploy a *stack* — a Compose file applied across the whole cluster, using `docker stack deploy` instead of `docker compose up`.

```
yaml

# docker-compose.yml (Note: for Swarm, only "version 3.x" Compose syntax is supported)
version: "3.8"

services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    deploy:
      # <-- "deploy" key ONLY works with docker stack
      #    it's IGNORED by plain `docker compose up`
      replicas: 3
      restart_policy:
        condition: on-failure # restart containers only if they crash (not on failure)
    resources:
      limits:
        cpus: "0.5" # max half a CPU core per replica
        memory: 256M # max 256MB RAM per replica
      reservations:
        cpus: "0.25" # guarantee at least this much is reserved for this service
        memory: 128M
    update_config:
      parallelism: 1
      delay: 10s
    networks:
      - my-overlay-net

  api:
    image: my-api-image:latest
    deploy:
      replicas: 2
    networks:
      - my-overlay-net

networks:
  my-overlay-net:
    driver: overlay # must be overlay driver for multi-node Swarm
```

```
bash
```

```
# Deploy the whole stack (all services in the file) to the swarm, under a stack
docker stack deploy -c docker-compose.yml my-stack
```

```
# -c docker-compose.yml : specify the compose file
# my-stack                : name for this stack; all resources get prefixed my-s
```

```
# List running stacks
docker stack ls
```

```
# See services belonging to a stack
docker stack services my-stack
```

```
# See individual running tasks/containers of a stack
docker stack ps my-stack
```

```
# Tear down an entire stack (all its services, networks) in one command
docker stack rm my-stack
```

Important beginner note: `docker compose up` (single host) and `docker stack deploy` (swarm) use *similar but not identical* Compose syntax. Things like `build:`, `depends_on` health-based ordering, and `.env` file interpolation behave differently or aren't supported in swarm stacks. Swarm expects pre-built images (usually from a registry), it does **not** build images for you.

STEP 10: Secrets and Configs (Hands-on, Intermediate-Advanced)

Never put passwords/API keys directly in images or plain environment variables in production. Swarm has built-in encrypted secret storage.

```
bash
```

```
# Create a secret from a file
echo "SuperSecretPassword123" | docker secret create db_password -
```

```
# Or from a file directly
docker secret create db_password ./password.txt
```

```
# List secrets (values are never shown again after creation, by design)
docker secret ls
```

```
bash
```

```
# Use it in a service – the secret is mounted as a FILE inside the container
# at /run/secrets/db_password, in memory (tmpfs), not written to disk
docker service create \
  --name my-db \
  --secret db_password \
  mysql:8
```

In your Compose stack file:

```
yaml
```

```
services:
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD_FILE: /run/secrets/db_password # app reads the file
    secrets:
      - db_password

secrets:
  db_password:
    external: true # means: this secret already exists (created via `dock
                  # don't try to create it from the compose file itself
```

Configs work almost identically but for non-sensitive config files (not encrypted at rest the same way secrets are):

```
bash
```

```
docker config create nginx_config ./nginx.conf

docker service create \
  --name my-web \
  --config source=nginx_config,target=/etc/nginx/nginx.conf \
  nginx:latest
```

STEP 11: Placement Constraints & Node Labels (Hands-on, Advanced)

Sometimes you need control over *where* replicas run — e.g., a database should only run on nodes with SSD storage, or GPU workloads only on GPU nodes.

```
bash
```

```
# Add a custom label to a node
```

```
docker node update --label-add storage=ssd worker-1
```

```
# See labels on a node
```

```
docker node inspect worker-1 --pretty
```

```
bash
```

```
# Constrain a service to only run on nodes with that label
```

```
docker service create \  
  --name my-db \  
  --constraint 'node.labels.storage==ssd' \  
  mysql:8
```

```
# Built-in constraints also exist, e.g., restrict to manager nodes only:
```

```
docker service create \  
  --name monitoring \  
  --constraint 'node.role==manager' \  
  prometheus:latest
```

In a Compose stack file:

```
yaml
```

```
services:  
  db:  
    image: mysql:8  
    deploy:  
      placement:  
        constraints:  
          - node.labels.storage==ssd  
        preferences:  
          - spread: node.labels.zone    # spread replicas evenly across a label
```

STEP 12: Health Checks and Global Services (Hands-on, Advanced)

```
bash
```

```
# A "global" service runs exactly ONE replica on EVERY node (great for monitoring  
# agents, log collectors – things you want on every machine)  
docker service create \  
  --name node-exporter \  
  --mode global \  
  prom/node-exporter
```

Health checks let Swarm know a container is *actually* working, not just "running":

```
yaml
```

```
services:  
  web:  
    image: my-app:latest  
    healthcheck:  
      test: ["CMD", "curl", "-f", "http://localhost/health"] # command to check  
      interval: 30s      # run this check every 30 seconds  
      timeout: 5s        # fail if it doesn't respond within 5 seconds  
      retries: 3          # mark unhealthy after 3 consecutive failures  
      start_period: 10s  # grace period after container starts before counting failures
```

If a container becomes "unhealthy," Swarm will kill and restart it automatically as part of maintaining desired state.

STEP 13: Draining Nodes for Maintenance (Hands-on, Advanced)

Before rebooting/patching a physical machine, you want Swarm to move its workloads elsewhere gracefully — this is called **draining**.

```
bash
```

```
# Set a node's availability to "drain" – Swarm evacuates all tasks off it and  
# won't schedule new ones there
```

```
docker node update --availability drain worker-2
```

```
# Do your maintenance (patch OS, reboot, upgrade Docker, etc.)
```

```
# Bring it back into service afterward
```

```
docker node update --availability active worker-2
```

```
# Availability options:
```

```
#   active  -> can receive new tasks (normal state)
```

```
#   pause   -> keeps running existing tasks, but won't get NEW ones
```

```
#   drain   -> existing tasks are moved off, and no new ones scheduled
```

STEP 14: Cluster Resilience — Losing a Manager (Theory, Advanced)

This is where the Raft consensus concept (Step 2) matters practically.

- With **1 manager**: if it dies, the cluster is unmanageable until it's restored (though running services keep running — workers keep executing existing tasks).
- With **3 managers**: cluster tolerates 1 manager failure and stays fully operational (needs majority = 2 out of 3 alive).
- With **5 managers**: tolerates 2 failures.
- Never go above 7 managers — more managers = more Raft consensus network overhead, with diminishing safety benefit.
- **Never run an even number of managers** (e.g., 4) — you get worse fault tolerance than the odd number below it (4 managers can only tolerate 1 failure too, same as 3, but with more overhead).

```
bash
```

```
# Always back up manager swarm state (needed for full disaster recovery)
```

```
# The raft logs live under /var/lib/docker/swarm on manager nodes – back this u
```

STEP 15: Leaving/Removing the Swarm (Hands-on)

```
bash
```

```
# On a WORKER node, leave the swarm cluster
```

```
docker swarm leave
```

```
# On a MANAGER node, you must force it (extra safety check since managers hold
```

```
docker swarm leave --force
```

```
# From another manager, remove a node that's already offline/left
```

```
docker node rm <NODE-NAME>
```

Quick Reference Cheat Sheet

```
bash
```

```
docker swarm init # create a swarm, become manager
```

```
docker swarm join-token worker # get command to add a worker
```

```
docker swarm join-token manager # get command to add a manager
```

```
docker node ls # list all nodes
```

```
docker node promote/demote <node> # change node role
```

```
docker node update --availability drain <node> # maintenance mode
```

```
docker service create --name x --replicas N image # create a service
```

```
docker service ls # list services
```

```
docker service ps <service> # list tasks/replicas of a service
```

```
docker service scale x=N # scale replicas
```

```
docker service update --image img:tag x # rolling update
```

```
docker service rollback x # undo last update
```

```
docker service rm x # delete service
```

```
docker stack deploy -c file.yml stackname # deploy a full compose stack
```

```
docker stack services stackname # list services in a stack
```

```
docker stack ps stackname # list tasks in a stack
```

```
docker stack rm stackname # remove entire stack
```

```
docker secret create name - # create a secret
```

```
docker config create name file # create a config
```

```
docker network create --driver overlay name # multi-host network
```

Common Beginner Mistakes

1. **Using `docker run` inside a swarm and expecting scaling/self-healing.** `docker run` creates a plain standalone container — Swarm doesn't manage it, won't restart it, won't load-balance it. You must use `docker service create` or `docker stack deploy`.
 2. **Forgetting `deploy:` only works with `docker stack deploy`, not `docker compose up`.** Beginners write a stack file with `replicas: 3` and run `docker compose up`, then wonder why only 1 container starts — `deploy:` is silently ignored by plain Compose.
 3. **Expecting Swarm to build images from a `build:` context.** Swarm doesn't build images — it only pulls pre-built images from a registry. You must `docker build` and `docker push` your image somewhere all nodes can pull it from (Docker Hub, a private registry, etc.) *before* deploying.
 4. **Running an even number of manager nodes** (commonly 2 or 4), thinking "more is better." This doesn't improve fault tolerance over the next-lower odd number and increases risk of a split-brain/no-quorum situation.
 5. **Not exposing the right ports between nodes.** Swarm needs specific ports open between all nodes: **TCP 2377** (cluster management), **TCP/UDP 7946** (node discovery), **UDP 4789** (overlay network data). Forgetting to open these on a firewall/security group is probably the #1 cause of "nodes won't join" issues.
 6. **Confusing `docker service scale` replicas with high availability.** Running 3 replicas of a stateless web app is safe. Running 3 replicas of a database that isn't designed for clustering (like a single MySQL instance) will corrupt data — Swarm doesn't magically make stateful apps distributed-safe.
 7. **Putting secrets in environment variables in the Compose file** instead of using `docker secret`. Environment variables are visible via `docker inspect` to anyone with access to the host — not encrypted or access-controlled like proper secrets.
 8. **Not using `--advertise-addr` on multi-NIC machines**, causing the swarm to advertise an internal/private IP that other nodes (especially on different networks) can't reach.
 9. **Assuming `docker stack rm` cleans up volumes.** Named volumes attached to stack services are NOT removed automatically — this is intentional (to protect data), but beginners are often confused why old data "reappears" when redeploying a stack.
 10. **Ignoring `update_config` and doing a big-bang `--replicas 0` then back to N**, instead of using `docker service update` for rolling updates — this causes real downtime instead of the zero-downtime rollout Swarm is designed to provide.
-

Interview Questions & Answers (Intermediate → Advanced)

Q1: What is the difference between a Docker container and a Docker Swarm service? A container is a single running instance you manage directly (`docker run`). A service is a declarative *desired state* managed by the swarm — you say "I want N replicas of this image," and the swarm continuously works to keep that many healthy containers (tasks) running across the cluster, rescheduling automatically if one fails.

Q2: Explain how Raft consensus is used in Docker Swarm. Manager nodes use the Raft algorithm to maintain a consistent, replicated log of the cluster's state (services, tasks, configs, etc.) across all managers. A leader is elected; all writes go through the leader and are only committed once a majority of managers acknowledge them. This ensures the cluster keeps a single consistent source of truth even if some managers fail, as long as a majority ("quorum") stays alive.

Q3: Why should you run an odd number of manager nodes? Because quorum requires a strict majority. With $2N+1$ managers, the cluster tolerates N failures. An even number (e.g., 4) tolerates the same number of failures as the next lower odd number (3) but adds network/coordination overhead without extra fault tolerance, and risks tie votes during leader election.

Q4: What is the Swarm routing mesh, and how does it work under the hood? It's Swarm's built-in Layer 4 load balancer. When a service publishes a port, Swarm opens that port on every node (not just ones running a replica) via the special `ingress` overlay network, and uses Linux's IPVS to route incoming connections to a healthy task anywhere in the cluster — even one on a different physical node. This means clients can hit any node's IP+port and get correctly routed.

Q5: How does Swarm perform a rolling update, and how do you control it? By default it updates replicas incrementally rather than all at once, based on `update_parallelism` (how many replicas at a time) and `update_delay` (pause between batches). You can also choose `update_order` (`stop-first` vs `start-first`) and configure `update_failure_action` to auto-rollback if too many replicas fail during the rollout, governed by `update_max_failure_ratio`.

Q6: What happens to running services if all manager nodes go down but worker nodes stay up? Already-running tasks on worker nodes keep running (workers execute independently once scheduled). However, the cluster becomes unmanageable — no new scheduling decisions, scaling, updates, or healing of failed tasks can happen until manager quorum is restored, since only managers hold and can act on the Raft-replicated cluster state.

Q7: Difference between `docker secret` and passing sensitive data via environment variables? Secrets are encrypted at rest in the Raft log, transmitted over mutual TLS, and mounted into containers as in-memory files (`/run/secrets/<name>`) only on nodes actually running that service's tasks — never written to the image or shown in `docker inspect`. Environment variables are plaintext, visible via `docker inspect`, and often end up logged or leaked, so they're unsuitable for real secrets.

Q8: What's the difference between "replicated" and "global" service modes? `replicated` (default) runs a specified number of task replicas, distributed across available nodes by the scheduler. `global` runs exactly one task on every node in the cluster (matching any placement constraints) — commonly used for node-level agents like log shippers or monitoring exporters.

Q9: How would you deploy a stateful service like a database in Swarm safely? Generally you'd either (a) avoid naive horizontal replication for single-instance databases and instead pin it to one node using placement constraints plus a node-affinity+volume strategy so it always restarts on the node holding its data, or (b) use a database engine explicitly designed for clustering (with its own replication protocol) and let Swarm just manage the container lifecycle, not the data consistency. Many teams instead run managed/external databases outside the swarm entirely for production stateful workloads.

Q10: What ports must be open between Swarm nodes, and why?

- TCP 2377 — cluster management/API communication between managers.
- TCP and UDP 7946 — node-to-node communication for the gossip-based membership/discovery protocol.
- UDP 4789 — VXLAN traffic for overlay network data between containers on different hosts. Blocking any of these typically causes join failures or broken inter-container networking.

Q11: What's the difference between `docker stack deploy` and `docker compose up` in terms of Compose file support? `docker stack deploy` only understands a subset of Compose keys relevant to cluster orchestration (`deploy:`, `secrets:`, `configs:`, `networks:` with `overlay` driver) and ignores/rejects things like `build:` — it expects pre-built images from a registry. `docker compose up` is for single-host development and supports build contexts, `depends_on` conditions, `.env` interpolation more fully, but has no concept of replicas, placement, or rolling updates.

Q12: How does Swarm decide which node to place a new task on? By default it uses a "spread" strategy that considers available resources (CPU/memory) and the number of tasks already on each node, aiming for balanced utilization. This can be influenced with `--constraint` (hard requirements, e.g., node labels/role) and `--placement-pref` (soft preferences, e.g., spreading evenly across a label like availability zone).

Q13: What's the difference between "drain" and "pause" node availability? `pause` stops a node from receiving *new* tasks but leaves its currently running tasks alone. `drain` actively evacuates all existing tasks off the node (rescheduling them elsewhere) in addition to blocking new ones — used before planned maintenance/shutdown.

Q14: Can Swarm services communicate with services outside the swarm (e.g., a legacy VM)? Yes, but not automatically via overlay networking (that's swarm-internal). You'd typically expose the service via a published port (through the routing mesh) that the external system connects to, or use a network bridge/VPN so the external host can route to the overlay subnet, though the latter is uncommon and more complex than just publishing ports.

Q15: Swarm vs Kubernetes — when would you choose Swarm in a real project? Swarm makes sense for smaller teams/clusters where operational simplicity matters more than Kubernetes' rich feature set (custom resource definitions, advanced autoscaling policies, huge ecosystem of operators/controllers). Since Swarm reuses Docker CLI/Compose syntax you already know, the learning curve and operational overhead are much lower — a reasonable trade for smaller, simpler deployments, while Kubernetes is generally preferred for large-scale, complex, multi-team production systems.

That covers Docker Swarm from first principles through production-grade concerns. The natural next step, once this feels solid, is exploring **Kubernetes** — many of the ideas here (nodes, replicas/pods, services, rolling updates, secrets, placement) map almost directly, just with more configuration surface area.